

Phase 1 — LangChain LCEL

Le moteur d'explication structurée : cinq concepts essentiels

Cours PyTutor2

Résumé

Ce document couvre les cinq briques fondamentales de la phase 1 du cours PyTutor2 : les `ChatPromptTemplate`, la sortie typée avec Pydantic et `with_structured_output`, la composition LCEL avec `|` et `.partial()`, l'exécution parallèle avec `RunnableParallel`, et le streaming token par token avec `.stream()`. Chaque concept est traité avec un énoncé, une architecture illustrée en TikZ, le code complet, des explications pas à pas et la vérification dans LangSmith.

Table des matières

1	ChatPromptTemplate : structurer un prompt en messages paramétrés	2
1.1	Solution	2
1.1.1	Architecture	2
1.1.2	Le code complet	2
1.1.3	Explications pas à pas	3
1.2	Vérification dans LangSmith	4
2	Sortie typée : schémas Pydantic et with_structured_output	5
2.1	Solution	5
2.1.1	Architecture	5
2.1.2	Le code complet	6
2.1.3	Explications pas à pas	6
2.2	Vérification dans LangSmith	8
3	LCEL : composition avec et figement de variables avec .partial()	9
3.1	Solution	10
3.1.1	Architecture	10
3.1.2	Le code complet	10
3.1.3	Explications pas à pas	11
3.2	Vérification dans LangSmith	12
4	RunnableParallel : exécuter plusieurs chaînes en concurrence	13
4.1	Solution	14
4.1.1	Architecture	14
4.1.2	Le code complet	14
4.1.3	Explications pas à pas	15
4.1.4	Exécution et observation	16
4.2	Vérification dans LangSmith	17
5	Streaming token par token avec .stream()	18
5.1	Solution	18
5.1.1	Architecture	18
5.1.2	Le code complet	19
5.1.3	Explications pas à pas	19

1 ChatPromptTemplate : structurer un prompt en messages paramétrés

Exercice 1. ChatPromptTemplate

On veut produire un assistant pédagogique Python dont le **ton et la profondeur** s'adaptent au niveau de l'élève (débutant, intermédiaire, avancé). Le concept à expliquer change à chaque requête, le niveau aussi.

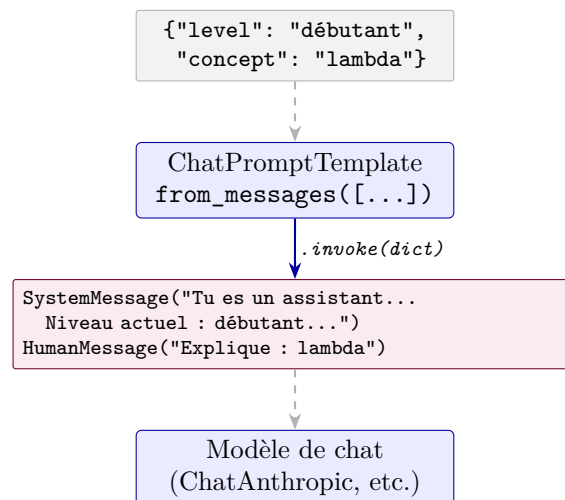
Contrainte : ne pas écrire trois prompts différents, ni concaténer des strings à la main. On veut *un seul* template réutilisable, avec deux variables `{level}` et `{concept}`, qui sera ensuite consommé par n'importe quel modèle compatible LangChain.

Objectifs pédagogiques

- Comprendre pourquoi un prompt moderne est une **liste de messages typés**, pas une chaîne de caractères.
- Maîtriser la séparation **system** / **human** et son rôle pour la qualité des réponses.
- Utiliser les **variables** `{nom}` d'un template et savoir ce qu'elles deviennent au moment du rendu.

1.1 Solution

1.1.1 Architecture



Un `ChatPromptTemplate` est essentiellement une **fonction** `dict → List[BaseMessage]`. Les variables `{level}` et `{concept}` sont substituées dans le contenu de chaque message au moment du rendu.

1.1.2 Le code complet

```

1 from langchain_core.prompts import ChatPromptTemplate
2
3 EXPLAIN_PROMPT = ChatPromptTemplate.from_messages([
4     (
5         "system",
6         "Tu es un assistant pédagogique pour des élèves apprenant Python. "
```

```

7         "Tu adaptes systématiquement ton vocabulaire et tes exemples au "
8         "niveau de l'élève. Niveau actuel : {level}.\n\n"
9         "Règles selon le niveau :\n"
10        " - 'débutant' : pas de jargon, exemples ultra-simples, "
11        "analogies du quotidien, pas plus de 10 lignes de code.\n"
12        " - 'intermédiaire' : tu peux supposer la syntaxe de base "
13        "connue, montre les usages idiomatiques (PEP 8, type hints).\n"
14        " - 'avancé' : montre les détails du modèle objet de Python, "
15        "les subtilités, les cas limites, les implications de performance."
16    ),
17    ("human", "Explique le concept suivant : {concept}"),
18 ])

```

1.1.3 Explications pas à pas

Étape 1 — Pourquoi une *liste de messages* et pas un string

Les modèles de chat modernes (Claude, GPT-4, Gemini, etc.) ne reçoivent pas un prompt monolithique : ils reçoivent une **conversation structurée** sous forme de messages typés. Chaque message a un **role** (`system`, `user/human`, `assistant`) et un contenu. L'API du fournisseur les traite différemment :

- Le **system** fixe la **personnalité** et les règles du jeu, et reçoit le plus de poids dans le respect des instructions.
- Les **human** sont les requêtes de l'utilisateur.
- Les **assistant** sont les réponses précédentes du modèle (utile pour du multi-tour, on les verra plus tard).

*Mettre les règles dans le **system** et la requête dans le **human** n'est pas cosmétique : les LLM modernes obéissent nettement mieux à une instruction présente dans le message système qu'à la même instruction noyée dans le message utilisateur. Si le ton de ton assistant dérive, c'est souvent que tu as mis tes règles au mauvais endroit.*

Étape 2 — La syntaxe `from_messages(...)`

`from_messages` accepte une liste de **tuples** de la forme (`role`, `content`) :

<code>("system", "...")</code>	devient un <code>SystemMessagePromptTemplate</code>
<code>("human", "...")</code>	devient un <code>HumanMessagePromptTemplate</code>
<code>("ai", "...")</code>	devient un <code>AIMessagePromptTemplate</code>
<code>("placeholder", "{x}")</code>	insère une liste de messages stockée dans <code>x</code>

`LangChain` transforme automatiquement chaque tuple en un objet `PromptTemplate` de la bonne classe, et l'ensemble compose un `ChatPromptTemplate`.

Étape 3 — Les variables `{nom}`

Toutes les accolades non-doublées dans une chaîne de message sont considérées comme des **variables** :

- `{concept}` dans le **human** attend une clé `concept` dans le dict d'entrée.
- `{level}` dans le **system** attend une clé `level`.
- Si une clé manque au moment du rendu, `LangChain` lève une `KeyError` explicite.

*Si tu veux écrire une vraie accolade dans le prompt (par exemple, demander au modèle de répondre en JSON), il faut la **doubler** : `{{ pour un { littéral, }}` pour un `}`. C'est le piège classique qui transforme un message en `KeyError: 'nom_de_clé_inattendu'` dès qu'on parle de structures JSON dans le prompt.*

Étape 4 — Le rendu via `.invoke()` ou `.format_messages()`



Deux façons d'appeler le template :

1. `EXPLAIN_PROMPT.invoke({"level": "débutant", "concept": "lambda"})` renvoie un `ChatPromptValue` (qu'on passe ensuite au modèle). C'est la voie LCEL, utilisée quand le template est dans une chaîne `prompt | model | ...`.
2. `EXPLAIN_PROMPT.format_messages(...)` renvoie directement la liste de `BaseMessage`. Pratique pour **déboguer** ou pour passer les messages à une API tierce.

*.format_messages(...) est le print du prompt engineer. Avant de te demander pourquoi le modèle répond bizarrement, imprime ce que **tu lui as réellement envoyé** — souvent l'erreur est là.*

1.2 Vérification dans LangSmith

Comment voir le prompt rendu

Quand le template est utilisé dans une chaîne et que LangSmith est activé, chaque appel au modèle produit un run `ChatAnthropic` (ou équivalent) dans lequel l'onglet *Input* contient **exactement les messages rendus** :

```
RunnableSequence
    ChatPromptTemplate
        Input : {level: "débutant", concept: "lambda"}
        Output : [SystemMessage(...), HumanMessage(...)]
    ChatAnthropic
        Input : la liste de messages ci-dessus
```

Les trois choses à regarder

1. L'onglet *Input* du nœud `ChatPromptTemplate` : tu vois le dict d'entrée brut. C'est là que tu repères une variable manquante ou mal typée.
2. L'onglet *Output* du même nœud : la liste complète des messages, variables substituées, prêts à partir au LLM. C'est la **vérité terrain** de ce que le modèle reçoit.
3. L'onglet *Input* du nœud `ChatAnthropic` : en principe identique à l'output du template ci-dessus. Si tu y vois autre chose, c'est qu'un `Runnable` intermédiaire a modifié les messages.

*Change la valeur de `level` entre "débutant" et "avancé" pour le même concept. Dans LangSmith, tu vois **la même structure de messages**, seul le contenu du `system` diffère. C'est la visualisation directe de la paramétrisation par template.*

Récapitulatif

Ce qu'il faut retenir :

1. Un prompt de chat est une **liste de messages typés**, pas un `string`. Le rôle (`system/human/ai`) change radicalement la façon dont le modèle traite le contenu.
2. `ChatPromptTemplate.from_messages([...])` est la voie idiomatique pour construire ce prompt. Les tuples (`role, content`) sont la forme la plus lisible.

3. Les **variables** {nom} dans le contenu deviennent des clés attendues du dict d'entrée. Doubler les accolades pour les accolades littérales.
4. `prompt.invoke(dict)` renvoie un `ChatPromptValue` pour la chaîne LCEL ; `prompt.format_messages(dict)` renvoie la liste brute, indispensable pour le débogage.
5. Mettre les règles dans le **system**, la requête dans le **human**. C'est l'erreur #1 qui fait dériver le ton d'un assistant.

2 Sortie typée : schémas Pydantic et `with_structured_output`

Exercice 2. Sortie typée avec Pydantic

Un assistant pédagogique ne doit pas répondre par un pavé de texte markdown. On veut un objet manipulable : une *explication structurée* contenant l'intuition, un exemple de code, un piège fréquent et une liste de questions de compréhension. Chaque champ a une sémantique précise, chaque question a sa réponse attendue.

Contrainte : la réponse doit être *immédiatement utilisable* en Python, sans étape de parsing fragile. `result.intuition` doit fonctionner ; `result.comprehension_questions[0].expected_answer` aussi.

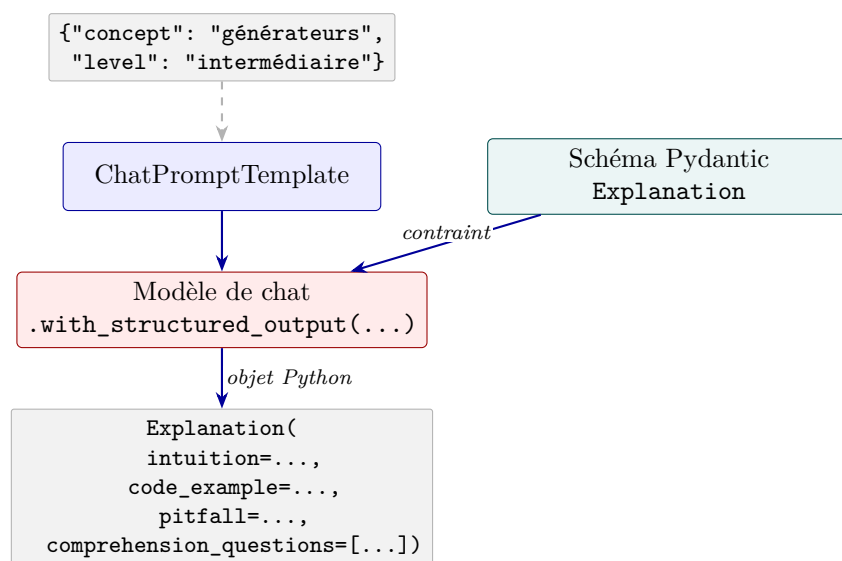
Objectifs pédagogiques

- Comprendre pourquoi exiger une **sortie structurée** d'un LLM change la nature du code qu'on écrit autour.
- Décrire un schéma de réponse avec `pydantic.BaseModel` et `Field(description=...)`.
- Maîtriser `model.with_structured_output(Schema)` et savoir ce qui se passe **sous le capot** (tool-calling).
- Reconnaître les cas où il faut *imbriquer* des sous-schémas.

2.1 Solution

2.1.1 Architecture

Vue d'ensemble



Le schéma Pydantic est **injecté dans l'appel** au modèle. Le SDK Anthropic l'expose comme un *tool* que Claude doit appeler ; la réponse en JSON est ensuite désérialisée vers un vrai objet Python.

2.1.2 Le code complet

```
1 from pydantic import BaseModel, Field
2
3 class ComprehensionQuestion(BaseModel):
4     """Une question de compréhension pour vérifier l'acquisition."""
5
6     question: str = Field(description="La question posée à l'élève.")
7     expected_answer: str = Field(
8         description="La réponse attendue, en une à deux phrases."
9     )
10
11
12 class Explanation(BaseModel):
13     """Réponse pédagogique complète sur un concept Python."""
14
15     concept: str = Field(description="Le nom du concept expliqué.")
16     intuition: str = Field(
17         description=(
18             "Explication intuitive en exactement 3 phrases. "
19             "Évite le jargon, vise la clarté pour l'élève."
20         )
21     )
22     code_example: str = Field(
23         description=(
24             "Un exemple de code Python minimal et commenté qui illustre "
25             "le concept. Préfère des noms de variables parlants."
26         )
27     )
28     pitfall: str = Field(
29         description="Un piège classique que rencontrent les élèves sur ce concept."
30     )
31     comprehension_questions: list[ComprehensionQuestion] = Field(
32         description="Exactement 3 questions de compréhension, de difficulté croissante."
33     )
34
35
36 # Utilisation dans une chaîne LCEL
37 def explain_chain_for_level(level: str):
38     model = get_model().with_structured_output(Explanation)
39     return EXPLAIN_PROMPT.partial(level=level) | model
```

2.1.3 Explications pas à pas

Étape 1 — Pourquoi structurer la sortie

Sans schéma, un LLM répond en texte libre. Le code applicatif doit alors *parser* cette réponse : regex, json.loads, conventions de markdown... Chaque appel devient un pari sur le fait que le modèle respectera bien le format demandé. En production, ce pari échoue régulièrement : virgule manquante, guillemets inversés, balise oubliée.

*Faire écrire le format au modèle est moins fiable que le lui imposer. La sortie structurée renverse la responsabilité : ce n'est plus le modèle qui « doit essayer de respecter » un format ; c'est l'API qui **refuse** toute sortie qui ne le respecte pas. Le code applicatif récupère un objet typé, point.*

Étape 2 — Le schéma comme *contrat* de réponse

Un schéma Pydantic n'est pas un simple type : c'est une description **lisible par le LLM**. Quand on appelle `with_structured_output(Explanation)`, LangChain transforme la classe en une *spécification de tool* qui contient :

- Le nom du schéma (`Explanation`).
- Le `docstring` de la classe (« Réponse pédagogique complète sur un concept Python »).
- Pour chaque champ : son nom, son type, et surtout sa `description=`.

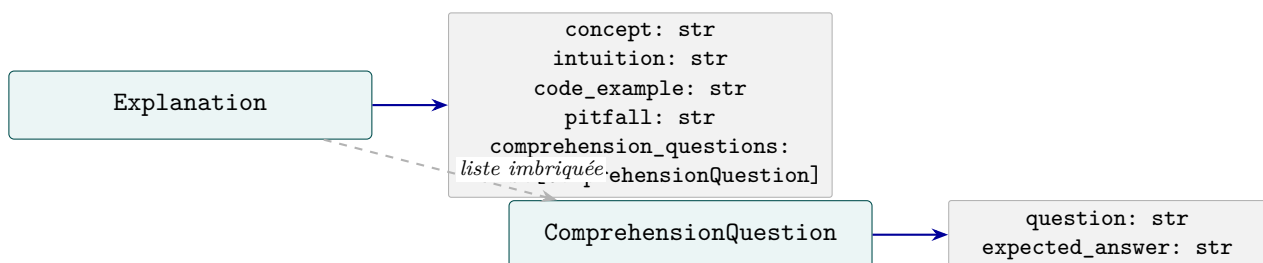
*La `description=` de chaque `Field` est une **partie du prompt**. Le modèle la lit pour décider quoi mettre dans le champ. Soigne-la au moins autant que ton message système.*

Étape 3 — Les types qui parlent au modèle

<code>str</code>	champ texte libre
<code>int, float</code>	nombre, validation automatique
<code>bool</code>	vrai / faux
<code>Literal["a", "b", "c"]</code>	ensemble fermé de valeurs (= énumération)
<code>list[X]</code>	liste typée (peut être un sous-schéma)
<code>X None</code>	champ optionnel
<code>BaseModel</code> imbriqué	sous-objet typé (cf. <code>ComprehensionQuestion</code>)

Tous ces types sont traduits en JSON Schema. Le LLM reçoit ce schéma et sait, par exemple, qu'un champ `Literal` n'a que 3 valeurs possibles — il ne pourra *physiquement pas* en renvoyer une quatrième.

Étape 4 — L'imbrication de schémas

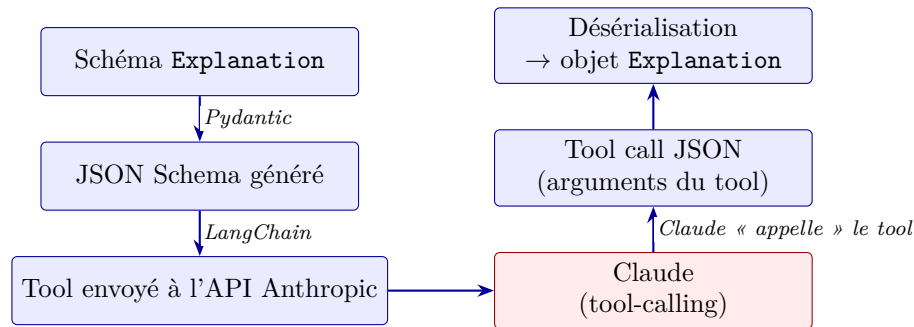


`Explanation` contient une `list[ComprehensionQuestion]`. Chaque question dans la liste est elle-même un objet typé, qu'on peut itérer en Python :

```
1 for q in result.comprehension_questions:
2     print(q.question)           # str
3     print(q.expected_answer)    # str
```

*Les sous-schémas sont la voie **idiomatique** pour modéliser des listes d'objets (questions, étapes, sources, citations...). Évite de stocker du JSON sérialisé dans un champ `str` — tu perds la validation et l'auto-complétion.*

Étape 5 — Ce qui se passe sous le capot (tool-calling)



`with_structured_output` n’invente pas un parseur de réponse texte. Il déclare un *tool* virtuel auprès du modèle, dont les arguments **sont** les champs du schéma. Claude est alors quasi-forcé d’« appeler » ce *tool* ; les arguments arrivent en JSON valide, validés par Pydantic, et reconvertis en objet Python.

Si tu utilises un modèle qui ne supporte pas le tool-calling natif (certains modèles open-source), `with_structured_output` retombe sur un parsing texte plus fragile. Vérifie toujours que le modèle ciblé supporte les tools — c’est le cas de tous les Claude messages, de GPT-4, et de Gemini.

Étape 6 — L’objet de sortie côté Python

```
1 chain = explain_chain_for_level("intermédiaire")
2 result: Explanation = chain.invoke({"concept": "les générateurs Python"})
3
4 # result est un VRAI objet Python.
5 # - PyCharm autocomplète result.intuition, result.pitfall, etc.
6 # - mypy / Pyright valident les accès.
7 # - On peut le sérialiser avec result.model_dump_json(indent=2).
8 print(result.intuition)
9 for q in result.comprehension_questions:
10     print(f"- {q.question}")
```

Le `result: Explanation` en annotation de type est important. Sans elle, l’IDE pense que `result` est de type `Any` et tu perds l’auto-complétion sur les champs. C’est le cadeau de la sortie structurée — sois sûr de l’utiliser.

2.2 Vérification dans LangSmith

L’arbre d’exécution attendu

```
RunnableSequence
  ChatPromptTemplate
    ChatAnthropic (structured_output=Explanation)
      Input : List[BaseMessage]
      Output : Explanation(...)
      Tool call : Explanation({...})
```


Les trois choses à regarder

1. **Onglet *Tool calls* du nœud ChatAnthropic** : tu y vois le tool **Explanation** déclaré, avec son JSON Schema complet (les types et les **description=** de chaque champ). C'est la **preuve** que ton schéma est bien transmis au modèle.
2. **Onglet *Output*** : la réponse brute du modèle est un tool-call dont les **args** contiennent les champs du schéma. LangChain la transforme automatiquement en objet **Explanation**.
3. **Onglet *Costs*** : noter que la sortie structurée coûte légèrement plus cher en tokens d'entrée (le schéma est envoyé) mais souvent moins en tokens de sortie (le modèle ne génère pas de préambule du genre « Voici votre réponse en JSON... »).

*Modifie une **description=** d'un champ et relance. Tu vois dans LangSmith que la nouvelle description est immédiatement présente dans le tool envoyé au LLM, et que le contenu du champ change conformément. C'est le prompt engineering appliqué directement au schéma de sortie.*

Récapitulatif

Ce qu'il faut retenir :

1. **Sortie structurée** = on impose au LLM un schéma de réponse, on récupère un objet Python typé, plus de parsing fragile.
2. **pydantic.BaseModel + Field(description=...)** est la voie standard. Les descriptions sont lues par le modèle, elles font partie du prompt.
3. **model.with_structured_output(Schema)** fonctionne via le **tool-calling** natif des API modernes — donc fiable, sans ré-essais ni parseur custom.
4. Types puissants à connaître : **Literal[...]** pour les énumérations, **BaseModel** imbriqué pour les sous-objets, **list[Schema]** pour les listes typées.
5. Annoter **result: MonSchema** dans le code applicatif pour bénéficier de l'auto-complétion et de la vérification statique.

3 LCEL : composition avec | et figement de variables avec .partial()

Exercice 3. Composition LCEL avec | et .partial()

On dispose d'un prompt unique **EXPLAIN_PROMPT** qui prend deux variables **{level}** et **{concept}**. On veut produire **trois chaînes spécialisées** (débutant, intermédiaire, avancé), chacune n'attendant plus que **{concept}** en entrée — le niveau étant déjà figé.

Contrainte : pas de duplication du prompt, pas de fonction **if/elif** qui réécrit la chaîne à chaque appel. La spécialisation doit se faire **à la construction**, et chaque chaîne produite doit rester un **Runnable** composable, traçable et streamable.

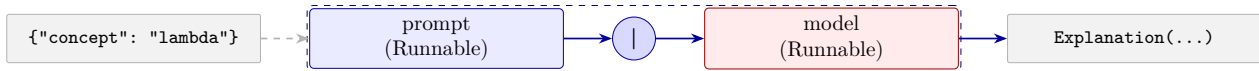
Objectifs pédagogiques

- Comprendre LCEL (*LangChain Expression Language*) comme une **algèbre de composition** de **Runnable**.
- Maîtriser l'opérateur **|** et le type qu'il produit.
- Utiliser **.partial()** pour figer une variable d'un prompt et produire des **chaînes spécialisées sans dupliquer de code**.
- Reconnaître le pattern *fabrique* (*factory*) pour construire dynamiquement des chaînes paramétrées.

3.1 Solution

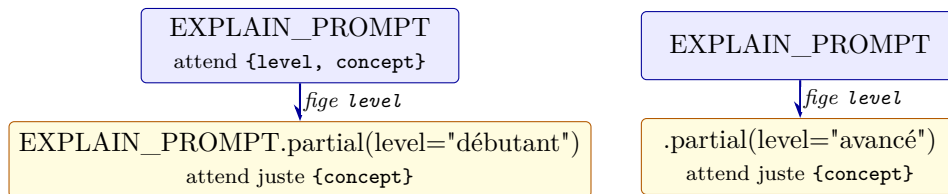
3.1.1 Architecture

La composition par |



`prompt | model` crée un nouveau `Runnable` qui chaîne les deux. La sortie du gauche devient l'entrée du droit. C'est typé, c'est composable, c'est traçable.

Le figement par `.partial()`



`.partial()` ne mute pas le prompt d'origine : il renvoie une **nouvelle** version partiellement renseignée. C'est l'équivalent de `functools.partial` pour les prompts.

3.1.2 Le code complet

```
1 from langchain_core.prompts import ChatPromptTemplate
2 from config import get_model
3
4 EXPLAIN_PROMPT = ChatPromptTemplate.from_messages([
5     ("system", "Tu es un assistant pédagogique Python. "
6         "Niveau actuel : {level}. "),
7     ("human", "Explique : {concept}"),
8 ])
9
10 def explain_chain_for_level(level: str):
11     """Renvoie une chaîne dont le niveau est figé.
12
13     `.partial(level=...)` crée un nouveau prompt avec une variable fixée.
14     On peut donc construire des chaînes spécialisées sans dupliquer le template.
15     """
16     model = get_model().with_structured_output(Explanation)
17     return EXPLAIN_PROMPT.partial(level=level) | model
18
19
20 # Trois chaînes spécialisées, un seul template
21 beginner_chain = explain_chain_for_level("débutant")
22 intermediate_chain = explain_chain_for_level("intermédiaire")
23 advanced_chain = explain_chain_for_level("avancé")
24
25 # Chaque chaîne n'attend plus que {concept}
26 result = intermediate_chain.invoke({"concept": "les générateurs Python"})
```

3.1.3 Explications pas à pas

Étape 1 — Qu'est-ce qu'un Runnable

LCEL repose sur une **interface unique** : tout objet manipulable est un `Runnable`. Cette interface garantit :

- `.invoke(input)` — exécution synchrone, un input.
- `.batch([inputs])` — exécution synchrone, plusieurs inputs.
- `.stream(input)` — itérateur, pour le streaming.
- `.ainvoke`, `.abatch`, `.astream` — versions asynchrones.

`ChatPromptTemplate`, un modèle, un parseur, un schéma — tous sont des `Runnable`. C'est ce qui permet de les composer librement.

Un seul protocole pour tout. C'est l'innovation de LCEL par rapport aux versions antérieures de LangChain : au lieu d'avoir des `LLMChain`, `SequentialChain`, `RouterChain`, ..., il n'y a qu'un type composable. Le code applicatif n'a plus à connaître qu'un seul vocabulaire.

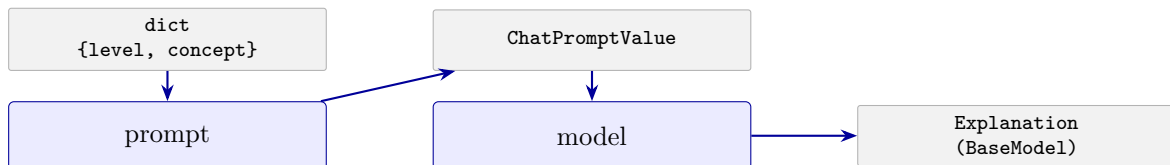
Étape 2 — Le `|` comme sucre syntaxique

`a | b` appelle `a.__or__(b)`, méthode définie sur la classe de base `Runnable`. Elle retourne une instance de `RunnableSequence(a, b)`. Mentalement, la transformation est :

$$\text{prompt} \mid \text{model} \quad \equiv \quad \text{RunnableSequence}(\text{prompt}, \text{model})$$

Et `a | b | c` compose en chaîne (associatif à gauche) — équivalent à `RunnableSequence(a, b, c)`. Quand on `.invoke(x)` le résultat, LCEL fait `c.invoke(b.invoke(a.invoke(x)))`.

Étape 3 — Le contrat de types entre maillons



La sortie d'un maillon doit être acceptable par le suivant. Le modèle accepte indifféremment un `ChatPromptValue`, une liste de messages ou un string — c'est ce qui fait que `prompt | model` marche naturellement.

Le `|` ne vérifie pas les types à la composition. Un mauvais chaînage (par exemple `model | prompt`) ne plantera qu'au `.invoke()`, avec une erreur souvent obscure. Compose toujours dans l'ordre entrée → traitement → sortie.

Étape 4 — `.partial()` : la fonction partielle des prompts

Un `ChatPromptTemplate` qui attend deux variables `{level}` et `{concept}` ressemble à une fonction Python `f(level, concept)`. `partial(level="débutant")` produit une nouvelle fonction `g(concept)` où `level` est figé.

```
1 # Le prompt d'origine attend les deux variables.
2 EXPLAIN_PROMPT.input_variables
3 # >>> ['level', 'concept']
4
```

```

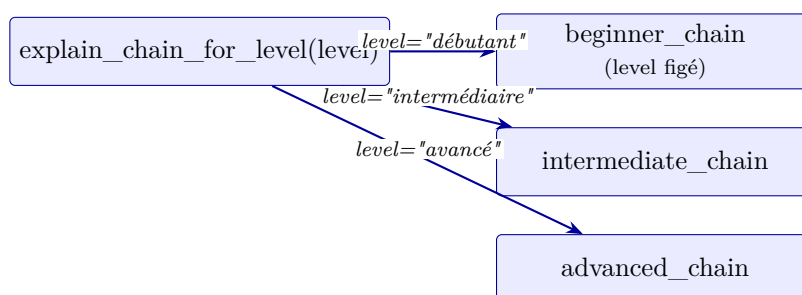
5 beginner_prompt = EXPLAIN_PROMPT.partial(level="débutant")
6 beginner_prompt.input_variables
7 # >>> ['concept']
8
9 # On peut désormais invoquer en ne passant que {concept}.
10 beginner_prompt.invoke({"concept": "lambda"})

```

.partial() ne mute pas le prompt original. C'est une opération pure qui renvoie une nouvelle instance. Tu peux donc appeler EXPLAIN_PROMPT.partial(...) cent fois sans casser le template de référence.

Étape 5 — Le pattern *fabrique*

`explain_chain_for_level(level)` est une **fabrique de chaînes** : on l'appelle avec un paramètre, elle nous renvoie un Runnable prêt à l'emploi, déjà configuré.



Avantages :

- **Pas de duplication** du prompt (un seul source de vérité).
- **Spécialisation à la construction**, pas à l'appel. Une fois construite, la chaîne ne demande plus le niveau à chaque `invoke`.
- **Composabilité conservée** : les chaînes spécialisées peuvent être passées à un `RunnableBranch`, un `RunnableParallel`, etc.

*On vient de remplacer trois fonctions différentes ou trois copies du prompt par **une fabrique paramétrée**. C'est l'esprit LCEL : construire dynamiquement des graphes d'exécution à partir de briques primitives, plutôt que d'écrire du code impératif.*

3.2 Vérification dans LangSmith

Que vois-tu dans l'arbre

Chaque chaîne produite par la fabrique est un `RunnableSequence`. Quand tu `invoke` `beginner_chain`, LangSmith trace :

```

RunnableSequence (beginner_chain)
  ChatPromptTemplate
    Input : {concept: "lambda"}
    Output : SystemMessage("...Niveau : débutant...")
  ChatAnthropic (structured_output=Explanation)

```

Le `level` n'apparaît plus dans l'input du run — il a été absorbé par le `.partial()`. C'est la preuve visuelle que le figement est bien effectif.

Les trois choses à regarder

1. Le **type** du run racine : `RunnableSequence`. C'est la matérialisation du `|`.
2. L'**input** du `ChatPromptTemplate` : seulement la (ou les) variable(s) non figée(s). Si tu y vois encore `level`, c'est que ton `.partial()` n'a pas été pris en compte.
3. L'**output** du `ChatPromptTemplate` : les messages complets avec `level` déjà remplacé. La substitution a eu lieu à la construction du template, pas dans le modèle.

Le tracing LangSmith permet de voir la différence entre « variable figée par `.partial()` » et « variable passée à `.invoke()` » : la première est invisible dans l'input du run, la seconde est visible. C'est utile pour déboguer les chaînes paramétrées.

Récapitulatif

Ce qu'il faut retenir :

1. LCEL repose sur **une seule interface** : `Runnable`. Tout objet manipulable (prompt, modèle, parseur, branch, parallel, ...) implémente `.invoke`, `.batch`, `.stream`, et leurs versions `async`.
2. `a | b` crée un `RunnableSequence(a, b)`. C'est purement du sucre syntaxique sur la méthode `Runnable.__or__`.
3. `ChatPromptTemplate.partial(...)` fige une ou plusieurs variables sans muter le template original. La chaîne produite n'attend plus que les variables restantes.
4. **Pattern fabrique** : une fonction `build_xxx(...)` qui renvoie un `Runnable` paramétré est la manière idiomatique de spécialiser à la construction plutôt qu'à l'appel.
5. Dans LangSmith, le `|` apparaît comme un nœud `RunnableSequence`; `.partial()` se voit à l'absence de la variable figée dans l'input du run.

4 RunnableParallel : exécuter plusieurs chaînes en concurrence

Exercice 4. RunnableParallel

On veut générer **deux versions** d'une même explication : une pour les débutants, une pour les intermédiaires. Les deux explications sont **indépendantes** l'une de l'autre (aucune n'a besoin du résultat de l'autre).

Contrainte de performance : chaque appel au LLM prend ~ 1 à 2 secondes. En exécution séquentielle, on attendrait ~ 2 à 4 secondes au total. On veut **ne payer que la latence d'un seul appel**, soit ~ 1 à 2 secondes — et un seul `invoke` côté code applicatif.

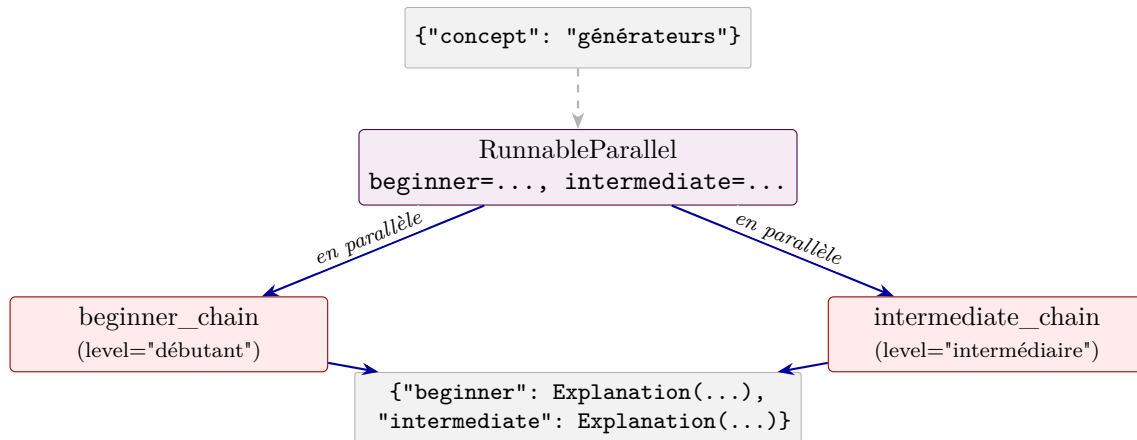
Objectifs pédagogiques

- Comprendre comment LCEL *parallélise* des chaînes indépendantes avec `RunnableParallel`.
- Quantifier le gain de latence par rapport à une boucle séquentielle.
- Distinguer `RunnableParallel` de `RunnablePassthrough.assign` (qui paraît proche mais a une sémantique différente).
- Repérer la structure « en éventail » dans LangSmith.

4.1 Solution

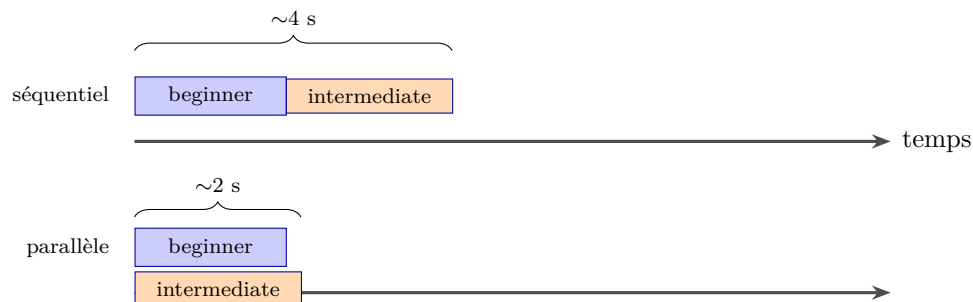
4.1.1 Architecture

Vue d'ensemble



`RunnableParallel` duplique l'input vers chaque branche, lance toutes les branches **en concurrence**, attend que toutes terminent, et retourne un **dict** dont les clés sont les noms de branches.

Diagramme temporel : séquentiel vs parallèle



Le gain est d'autant plus marqué qu'on a de branches : 3 chaînes en parallèle restent à la latence d'une seule chaîne (modulo le plafond de *rate limiting* du fournisseur).

4.1.2 Le code complet

```
1 from langchain_core.runnables import RunnableParallel
2
3 def explain_chain_for_level(level: str):
4     model = get_model().with_structured_output(Explanation)
5     return EXPLAIN_PROMPT.partial(level=level) | model
6
7
8 def build_multi_level_chain():
9     """Génère les versions débutant ET intermédiaire en un seul invoke."""
10    return RunnableParallel(
11        beginner=explain_chain_for_level("débutant"),
12        intermediate=explain_chain_for_level("intermédiaire"),
13    )
14
15
16 # Utilisation
17 multi = build_multi_level_chain()
18 results = multi.invoke({"concept": "les générateurs Python"})
```

```

19
20 # results est un dict.
21 # Les deux appels LLM ont été lancés en concurrence ; on n'a payé
22 # que la latence du plus lent.
23 print(results["beginner"].intuition)
24 print(results["intermediate"].intuition)

```

4.1.3 Explications pas à pas

Étape 1 — Le contrat de RunnableParallel

RunnableParallel prend un **ensemble de Runnable nommés**, généralement passés en kwargs :

```

1 RunnableParallel(
2     beginner=chain_a,
3     intermediate=chain_b,
4     advanced=chain_c,
5 )

```

À l'invocation :

1. L'input **x** est **dupliqué** vers chaque branche.
2. Toutes les branches s'exécutent en parallèle (via le pool de threads pour les chaînes synchrones, via l'évent loop asyncio pour les chaînes async).
3. LCEL attend que **toutes** terminent.
4. La sortie est un dict {**nom_branche**: **résultat**}.

Un seul input, plusieurs sorties. C'est le pattern « éventail » : on prend une question, on en demande plusieurs angles, et chaque branche travaille sur sa version.

Étape 2 — Le gain de latence n'est pas le gain de débit

Un appel LLM passe la quasi-totalité du temps à **attendre** la réponse du serveur. Sur le client, le thread est endormi. Lancer deux appels en parallèle ne consomme donc pas deux fois plus de CPU local : ça consomme deux *tickets* chez le fournisseur en même temps.

Mesure	Séquentiel	Parallèle (2 branches)
Latence totale	$T_1 + T_2$	$\max(T_1, T_2)$
Tokens consommés	$N_1 + N_2$	$N_1 + N_2$
Coût \$	idem	idem
CPU client	1 thread occupé	2 threads endormis sur I/O

*La parallélisation **ne réduit pas la facture** : tu paies toujours chaque token. Elle réduit uniquement la latence perçue par l'utilisateur. Pour un usage UI où le temps de réponse compte, c'est décisif. Pour un job batch nocturne, le gain est nul (et tu risques juste de saturer plus vite ton rate limit).*

Étape 3 — Le sucre syntaxique dict

RunnableParallel a deux formes équivalentes :

```

1 # Forme explicite
2 chain = RunnableParallel(beginner=a, intermediate=b)
3

```

```

4 # Forme dict (utile pour des noms non-identifieurs, ex. avec espaces)
5 chain = RunnableParallel({"beginner": a, "intermediate": b})

```

Et LCEL accepte même un **dict nu** dans une composition par `|` : il sera implicitement converti en `RunnableParallel`.

```

1 chain = some_prompt | {"answer": chain_a, "summary": chain_b}
2 # strictement équivalent à
3 chain = some_prompt | RunnableParallel(answer=chain_a, summary=chain_b)

```

Le **dict implicite** dans une chaîne LCEL signifie toujours `RunnableParallel`. Quand tu vois `{ "x": f, "y": g }` dans un `|`, lis « éventail vers `f` et `g`, sortie nommée ».

Étape 4 — `RunnableParallel` vs `RunnablePassthrough.assign`

Ces deux primitives se ressemblent, mais l'intention diffère :

<code>RunnableParallel</code>	<code>RunnablePassthrough.assign</code>
Renvoie un nouveau dict contenant uniquement les clés des branches.	Enrichit l'input existant : garde toutes les clés d'entrée <i>et</i> ajoute les nouvelles.
Sortie : <code>{nom1: ..., nom2: ...}</code> (les inputs disparaissent).	Sortie : <code>{**input, nom1: ..., nom2: ...}</code> (inputs préservés).
Usage typique : fan-out final (l'utilisateur veut <i>le résultat</i>).	Usage typique : ajout d'un champ calculé à mi-pipeline.

Si tu utilises `RunnableParallel` à la place de `.assign(...)` dans un pipeline, l'**input** d'origine **disparaît** dans la sortie. Les maillons suivants qui s'attendaient à lire la clé **concept** ne la trouveront plus. C'est l'un des bugs LCEL les plus fréquents.

Étape 5 — Limites et bonnes pratiques

- **Rate limits** : chaque branche compte. 5 branches en parallèle, c'est 5 appels simultanés chez le fournisseur — vérifie que tu ne dépasses pas le RPM (*requests per minute*) de ton compte.
- **Erreur partielle** : si une branche échoue, par défaut tout échoue. Pour tolérer les erreurs partielles, enveloppe chaque branche dans `.with_fallbacks([...])`.
- **Branches indépendantes uniquement** : si la branche B a besoin du résultat de la branche A, ce n'est pas du parallèle — c'est une séquence, à composer avec `|` et `.assign`.

4.1.4 Exécution et observation

Le code de l'exécuteur

```

1 import time
2
3 multi = build_multi_level_chain()
4
5 t0 = time.perf_counter()
6 results = multi.invoke({"concept": "les générateurs Python"})
7 elapsed = time.perf_counter() - t0
8
9 print(f"Durée RunnableParallel : {elapsed:.2f} s")
10
11 # Pour comparaison, version séquentielle

```



```

12 t0 = time.perf_counter()
13 r1 = explain_chain_for_level("débutant").invoke({"concept": "les générateurs Python"})
14 r2 = explain_chain_for_level("intermédiaire").invoke({"concept": "les générateurs
    ↪ Python"})
15 elapsed_seq = time.perf_counter() - t0
16
17 print(f"Durée séquentielle : {elapsed_seq:.2f} s")
18 print(f"Speedup : x{elapsed_seq / elapsed:.2f}")

```

Typiquement, on observe un speedup proche de $\times 1.8$ à $\times 2.0$ pour deux branches (le facteur 2 théorique étant légèrement entamé par l'overhead de *threading* et les variations de latence côté fournisseur).

4.2 Vérification dans LangSmith

L'arbre d'exécution attendu

```

RunnableParallel<beginner, intermediate>
    [beginner] RunnableSequence
        ChatPromptTemplate
        ChatAnthropic (structured_output) ← LLM #1
    [intermediate] RunnableSequence
        ChatPromptTemplate
        ChatAnthropic (structured_output) ← LLM #2

```

Les deux sous-séquences sont des **frères** sous le même `RunnableParallel`. LangSmith affiche un *timeline* où l'on voit clairement les deux barres se chevaucher temporellement.

Les quatre choses à regarder

1. **Timeline en haut du run racine** : deux barres parallèles. Si les barres sont en *série* (l'une après l'autre), c'est que `RunnableParallel` n'a pas été utilisé correctement (par exemple, tu as fait deux `invoke` séparés dans une boucle).
2. **Input des deux branches** : identique. C'est la duplication de l'input qu'on a évoquée.
3. **Output du run racine** : un dict avec les clés `beginner` et `intermediate`. Aucune autre clé.
4. **Onglet *Costs*** : les tokens et le \$ s'additionnent entre branches — la parallélisation ne change rien à la facture.

*Lancer deux appels LLM en parallèle dans LangChain, c'est littéralement **ajouter un mot** : `RunnableParallel`. Pas de `threads` à gérer, pas d'`asyncio.gather`, pas de `pool`. Le runtime LCEL fait tout, et le tracing LangSmith te montre la concurrence à l'œil nu.*

Récapitulatif

Ce qu'il faut retenir :

1. `RunnableParallel(nom1=chain1, nom2=chain2, ...)` duplique l'input vers chaque branche, exécute toutes les branches en **concurrence**, et renvoie un **dict** de résultats.
2. **Latence** : $\max(T_i)$ au lieu de $\sum T_i$. Coût en tokens **inchangé**. C'est un gain de latence, pas un gain de débit.
3. `prompt / {"a": chain_a, "b": chain_b}` est du sucre syntaxique : le dict est implicitement converti en `RunnableParallel`.

4. *Ne pas confondre avec `RunnablePassthrough.assign` : `Parallel` remplace l'input, `.assign` l'enrichit.*
5. *Dans `LangSmith`, la concurrence se voit immédiatement : deux barres de timeline qui se chevauchent sous le même nœud parent.*

5 Streaming token par token avec `.stream()`

Exercice 5. Streaming token par token

L'utilisateur pose une question au tuteur Python. Plutôt que d'attendre *toute* la réponse (souvent 2 à 5 secondes), on veut afficher la réponse **au fur et à mesure** qu'elle est générée, comme dans ChatGPT : le premier mot apparaît au bout de ~300 ms, puis les suivants s'écrivent en temps réel.

Contrainte : rester un `Runnable` LCEL classique, et afficher token par token dans le terminal, sans buffering perturbateur.

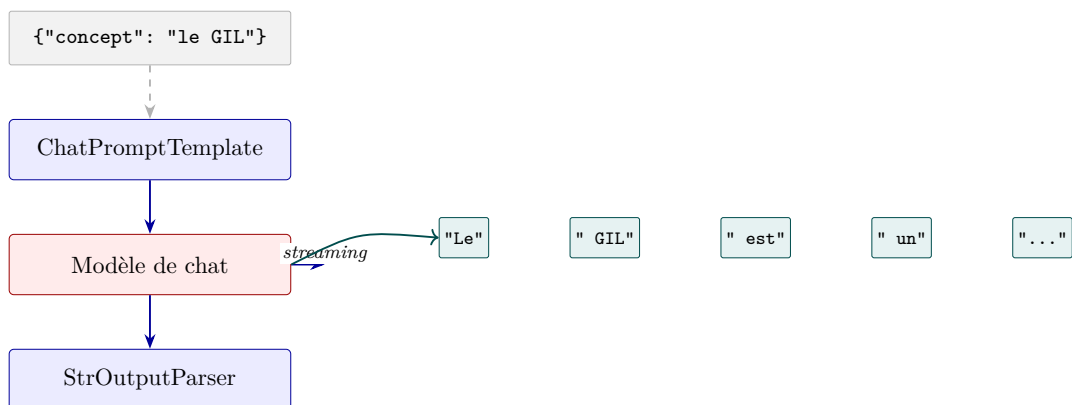
Objectifs pédagogiques

- Comprendre la différence entre `.invoke()` (réponse unique) et `.stream()` (itérateur de morceaux).
- Savoir construire une chaîne *streamable* et l'afficher au fil de l'eau dans un terminal ou une UI.
- Comprendre pourquoi `with_structured_output` ne stream pas bien et quand basculer sur `StrOutputParser`.
- Connaître les pièges classiques du streaming (buffering, flush, retour chariot).

5.1 Solution

5.1.1 Architecture

Vue d'ensemble



pour `chunk` in `chain.stream(...)` : `print(chunk, end="", flush=True)`

L'API du modèle ouvre une connexion *server-sent events* avec le fournisseur. Chaque token reçu est immédiatement émis par la chaîne. Tant qu'on consomme l'itérateur, les tokens arrivent.

5.1.2 Le code complet

```
1 from langchain_core.prompts import ChatPromptTemplate
2 from langchain_core.output_parsers import StrOutputParser
3 from config import get_model
4
5 STREAM_PROMPT = ChatPromptTemplate.from_messages([
6     ("system", "Tu es un assistant pédagogique Python concis et précis."),
7     ("human", "Explique en 2 paragraphes : {concept}"),
8 ])
9
10
11 def build_streaming_chain():
12     """Chaîne texte simple, idéale pour du streaming dans un terminal/UI."""
13     return STREAM_PROMPT | get_model() | StrOutputParser()
14
15
16 # Utilisation
17 streamer = build_streaming_chain()
18 for chunk in streamer.stream({"concept": "le GIL en Python"}):
19     print(chunk, end="", flush=True)
20 print() # saut de ligne final
```

5.1.3 Explications pas à pas

Étape 1 — .invoke() vs .stream()

.invoke(input)	.stream(input)
Renvoie une valeur unique : la réponse complète.	Renvoie un <i>itérateur</i> de morceaux (chunks).
Le code attend que tout soit prêt avant de continuer.	Le code itère et reçoit les morceaux dès qu'ils arrivent.
Latence perçue = latence totale.	Latence perçue = latence du <i>premier</i> token (300 ms).
Bien pour batch, scripts, données structurées.	Bien pour UI temps réel, terminal interactif.

Le *time-to-first-token (TTFT)* est ce qui rend une interface « vivante ». Même si la durée totale est identique, voir la réponse **commencer** à apparaître au bout de 300 ms change radicalement la sensation d'attente.

Étape 2 — Pourquoi StrOutputParser ?

Par défaut, un modèle de chat renvoie des AIMessageChunk (des fragments d'AIMessage). En streaming, on ne veut pas avoir à faire `chunk.content` dans la boucle d'affichage — on veut directement le *texte*. C'est exactement le rôle de `StrOutputParser` :

Entrée	Sortie
AIMessageChunk(content="Le")	"Le"
AIMessageChunk(content=" GIL")	" GIL"
...	...

```
1 # Sans StrOutputParser
2 for chunk in (STREAM_PROMPT | get_model()).stream({"concept": "X"}):
3     print(chunk.content, end="", flush=True) # accès manuel à .content
```

```

4
5 # Avec StrOutputParser
6 for chunk in (STREAM_PROMPT | get_model() | StrOutputParser()).stream({"concept": "X"}):
7     print(chunk, end="", flush=True) # le chunk EST une str

```

Étape 3 — Pourquoi with_structured_output streame mal



Un schéma Pydantic est **tout ou rien** : tant que tous les champs requis ne sont pas remplis, l'objet ne peut pas être validé. Le modèle peut bien streamer le JSON token par token, on ne peut pas *désérialiser* en cours de route — la moitié des champs manquent.

*Appeler `.stream()` sur une chaîne `with_structured_output` ne plantera pas, mais souvent la chaîne accumule tout en interne et n'émet qu'un **seul gros chunk** à la fin. Le bénéfice du streaming disparaît. Pour un streaming réel, soit on accepte du texte (`StrOutputParser`), soit on utilise des techniques avancées (parsing JSON incrémental, mais c'est une autre histoire).*

Étape 4 — La boucle d'affichage côté Python

```

1 for chunk in streamer.stream({"concept": "le GIL en Python"}):
2     print(chunk, end="", flush=True)
3 print()

```

Trois détails subtils mais importants :

- `end=""` : empêche `print` d'ajouter un `\n` après chaque chunk. Sans ça, chaque token est affiché sur sa propre ligne — illisible.
- `flush=True` : force l'écriture sur le terminal **immédiatement**. Python bufferise stdout par défaut (par bloc de 4 ko ou par ligne) ; sans flush, on voit la réponse arriver par paquets, pas par tokens.
- Le `print()` final est juste là pour le saut de ligne après toute la réponse.

`flush=True` est le secret du streaming visible. Si tu oublies, tu as l'impression que ton streaming ne marche pas — alors qu'il marche, mais c'est le terminal qui retient les caractères. C'est la question #1 posée en debug.

Étape 5 — Cas async : `.astream()`

Pour une UI web ou un endpoint FastAPI, on veut la version asynchrone :

```

1 async def stream_to_websocket(websocket, concept: str):
2     async for chunk in streamer.astream({"concept": concept}):
3         await websocket.send_text(chunk)

```

La sémantique est identique à `.stream()`, mais on utilise `async for` et chaque `await` libère l'évent loop pour les autres connexions. C'est le pattern à connaître pour servir N clients simultanément.

5.2 Vérification dans LangSmith

L'arbre d'exécution attendu

```
RunnableSequence
  ChatPromptTemplate
    ChatAnthropic (streaming=True)
      Tokens : "Le", " GIL", " est", " un", ...
        StrOutputParser
```

LangSmith reconstitue la réponse complète à la fin et l'affiche dans l'onglet *Output*, mais conserve la durée *end-to-end* et le *TTFT* dans les métriques.

Les trois choses à regarder

1. **Métrique *Time to first token*** : temps entre l'appel et le premier token reçu. C'est la métrique qui détermine la *sensation de vivacité* de ton UI. Souvent affichée sous `ttft_ms` dans les détails du run.
2. **Onglet *Output* du nœud `ChatAnthropic`** : l'`AIMessage` reconstitué (LangSmith concatène les chunks pour l'affichage). Tu ne vois *pas* chaque token séparément dans l'UI, mais la trace HTTP sous-jacente est bien du streaming.
3. **Type du modèle dans la trace** : `streaming=True` ou un flag équivalent. Si tu vois `streaming=False`, ton appel est en mode bloquant.

Compare deux runs côte à côte : l'un avec `.invoke`, l'autre avec `.stream`, sur le même prompt. Le total de tokens et le coût \$ sont identiques. Mais le run streamé a un TTFT $\sim 10\times$ plus court. C'est, mesurée, la valeur ajoutée du streaming.

Récapitulatif

Ce qu'il faut retenir :

1. `chain.stream(input)` renvoie un **itérateur** de morceaux ; `chain.invoke(input)` renvoie la réponse complète. Même chaîne, deux modes de consommation.
2. Pour streamer du texte, terminer la chaîne par `StrOutputParser` pour que chaque morceau soit déjà une `str` (et pas un `AIMessageChunk`).
3. `with_structured_output` ne streame pas vraiment : un schéma est tout-ou-rien, on ne peut pas désérialiser un objet partiel. Pour du streaming réel, accepter du texte.
4. Dans la boucle d'affichage : `print(chunk, end="", flush=True)`. Sans `flush=True`, les buffers du terminal masquent l'effet streaming.
5. Pour le web ou les apps `async`, utiliser `.astream()` avec `async for`. Sémantique identique, mais l'event loop reste libre pour les autres connexions.